

TATA STEEL · PREDICTIVE MAINTENANCE PLATFORM

Sherlock AI

Intelligent Maintenance Wizard for Steel Manufacturing Plants

Technical Documentation & System Design

An AI-driven decision-support platform that consolidates operational logs, live condition-monitoring data and engineer queries into explainable diagnosis, failure prediction, prioritized maintenance actions, and an interactive 3D digital twin of the plant.

SOLUTION

Autonomous Plant Intelligence System

CORE INNOVATION

Live 3D Digital Twin + LangGraph multi-agent reasoning

VERSION

1.0 — Executive Edition

01 System Architecture

Sherlock AI uses a decoupled three-tier architecture: a Next.js presentation tier, a FastAPI service tier exposing a versioned REST API, and an intelligence tier combining a LangGraph multi-agent reasoning engine with a real-time sensor-simulation and prediction engine. Historical knowledge is served from a pandas-backed data layer. The tiers are independently deployable — the frontend ships to an edge host (Vercel) and the backend runs as a container/web service (Render), communicating only over the REST contract.

PRESENTATION TIER

Next.js 14 · React · TypeScript · Tailwind · Three.js · Recharts — axios REST client

SERVICE TIER · REST API

FastAPI + Uvicorn · JWT auth · CORS · GZip · /auth /dashboard /maintenance /safety /energy /production /chat /reports /demo · OpenAPI at /api/docs

INTELLIGENCE TIER

LangGraph StateGraph · supervisor → 5 domain agents · Gemini 2.5 Flash (OpenRouter) · rule-based fallback

REAL-TIME ENGINE

live_sensor.py background thread (10s tick) · failure-probability model · threshold-breach alerting

DATA LAYER

data_service.py over CSV (plant · maintenance · energy · safety) via pandas · PDF report generation (reportlab)

DESIGN PRINCIPLE Each tier talks only over the REST contract, so any layer can be scaled, replaced, or redeployed independently without touching the others.

02

Technology Stack

A modern, production-grade stack — typed end-to-end, container-ready, and deployable to edge + cloud.

Frontend

Layer	Technology
Framework	Next.js 14 (App Router), React 18, TypeScript
Styling	Tailwind CSS (token-based theme), custom glassmorphism
3D / Digital Twin	Three.js (procedural geometry, OrbitControls, fog, lighting)
Charts & Data Viz	Recharts (line / area / bar / radar / pie)
Icons / HTTP / Dates	lucide-react, axios, date-fns

Backend

Layer	Technology
API Framework	FastAPI 0.111 + Uvicorn (ASGI), GZip + CORS middleware
AI Orchestration	LangGraph 0.2 (StateGraph supervisor + domain agents)
LLM	Google Gemini 2.5 Flash via OpenRouter API (httpx client)
Auth & Security	JWT (python-jose), passlib + bcrypt password hashing
Data & Compute	pandas, numpy (CSV historical data + KPI computation)
Reporting	reportlab + pypdf (structured PDF maintenance reports)
Persistence (optional)	SQLAlchemy + asyncpg (PostgreSQL-ready)
Config	pydantic-settings, python-dotenv (.env)

03 Data Flow & System Flow

Two data streams feed the platform. (1) Historical/operational data — equipment, maintenance, energy and safety records — is loaded from CSV files by `data_service.py` using pandas and aggregated into KPIs on demand. (2) A live condition-monitoring stream is produced by `live_sensor.py`, a background thread that updates sensor readings every 10 seconds and continuously recomputes failure probability and status.

Request lifecycle — load dashboard

- User authenticates; the frontend stores a token and attaches it to API calls.
- A page mounts and the axios client calls `GET /api/dashboard/kpis` and `/api/dashboard/timeseries`.
- FastAPI routes call `data_service` (historical KPIs) and `live_sensor` (live readings, events, KPI deltas).
- JSON is GZip-compressed and returned; React renders metrics, charts, the risk map and the 3D twin.
- The dashboard polls the live endpoint on an interval, so alerts and KPI deltas update without a reload.

AI query lifecycle — engineer asks a question

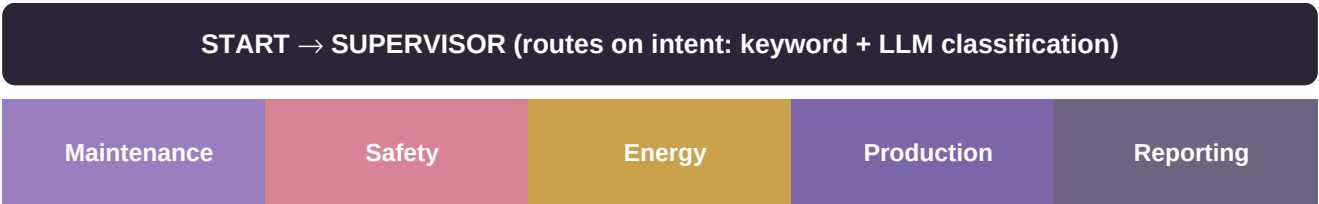
- Frontend posts the natural-language query to `POST /api/chat`.
- The LangGraph supervisor node classifies intent and routes to the correct domain agent.
- The agent assembles a context-grounded prompt (current sensor state + relevant records) and calls the LLM.
- A structured, explainable answer (diagnosis, actions, confidence, impact) is returned and rendered; follow-up turns retain context.

TWO STREAMS Historical CSV knowledge plus a live 10-second condition-monitoring stream feed one unified REST API contract.

04

Model Design & Reasoning Pipeline

The reasoning core is a LangGraph StateGraph implementing a supervisor / multi-agent pattern. A typed shared state flows from a single entry point through a supervisor that routes to one of five specialized agents, each responsible for a maintenance domain.



Supervisor routing

Intent is determined from a curated keyword table per domain: *vibration, bearing, lubrication, wear* → MAINTENANCE; *gas leak, hazard, DGFASLI, PPE* → SAFETY; *kWh, heat recovery, CO2* → ENERGY; *throughput, bottleneck, rolling mill* → PRODUCTION; *summary, executive, overview* → REPORTING. This keeps routing fast, transparent and debuggable.

LLM layer

Property	Configuration
Model	google/gemini-2.5-flash (served via OpenRouter)
Call style	httpx POST to /chat/completions, 30s timeout, temperature 0.3, max_tokens 1200
Grounding	Prompts injected with live sensor state and domain records for context-aware answers
Resilience	Deterministic rule-based fallback if no API key / API failure — app never breaks in a demo

EXPLAINABILITY Every recommendation is returned as diagnosis + prioritized actions (IMMEDIATE / 24h / 48h / 1 week) + a confidence percentage + an expected operational/financial impact — tied back to the sensor evidence that triggered it, so engineers can trace why an action is advised.

05 Alerting & Prediction Logic

A background thread (`live_sensor.py`) simulates condition monitoring for six critical assets (Blast Furnace 1/2, Rolling Mill, Coke Oven, Sinter Plant, BOF Converter). Every 10 seconds it random-walks vibration, temperature, pressure and production rate around realistic base values, then recomputes a failure-probability score.

Failure-probability model (transparent & physics-informed)

```
v_norm = vibration / 10.0          # normalise each signal
t_norm = (temperature - 200) / 1800
p_norm = pressure / 20.0

failure_probability = 0.35*v_norm + 0.40*t_norm + 0.25*p_norm (+ noise)
                    -> clamped to [0.01, 0.99]
```

Weights encode domain priority — temperature and vibration dominate thermal-mechanical failure modes. The score maps to status bands that drive alerting:

Status band	Condition	Meaning
NORMAL (green)	$fp \leq 0.50$	Healthy operating envelope
WARNING (amber)	$0.50 < fp \leq 0.75$	Degradation — schedule intervention
CRITICAL (red)	$fp > 0.75$	Imminent failure — act now

Real-time alerting

- Threshold-breach events are emitted the moment an asset crosses 60% failure probability (severity HIGH, or CRITICAL above 75%).
- A rolling buffer keeps the most recent events; the dashboard polls and surfaces them in the live alarm feed.
- Demo injection (`inject_demo_alert`) spikes a unit's sensors to showcase end-to-end reactive detection on stage.
- KPI deltas (safety, energy, production, downtime-risk) are computed live against baselines so trends move in real time.

Prediction / RUL

Degradation trend and remaining-useful-life estimates are derived from the failure-probability trajectory and surfaced per asset and in the digital twin's RUL view mode, enabling the shift from reactive to proactive maintenance.

06 Assumptions & Limitations

The current build is a faithful, demo-ready implementation. The following assumptions and boundaries define what is simulated today and what is engineered to plug in next.

Simulated telemetry

Live sensor values are generated by a realistic random-walk simulator rather than a physical OT/IoT bus; the architecture is designed to swap this for a real sensor feed (MQTT/OPC-UA) without UI changes.

Demo datasets

Historical records are representative CSV samples; some dashboard figures are illustrative for the demo.

Knowledge base

The current build reasons over operational data + LLM priors. A retrieval (RAG) layer over equipment manuals, SOPs and past work-orders is designed-for but not yet wired — it is the primary planned enhancement.

Feedback loop

Engineer confirm/correct feedback is on the roadmap; recommendations are currently generated per request.

LLM dependence

Online reasoning needs an OpenRouter key; without it the system falls back to deterministic, data-informed rule-based responses.

07 Installation, Configuration & Run

Prerequisites

- Python 3.11+ and Node.js 18+ (with npm).
- An OpenRouter API key (optional — the system runs with rule-based fallback if omitted).

Backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate          # Windows (source .venv/bin/activate on macOS/Linux)
pip install -r requirements.txt

# create backend/.env (do NOT commit it):
#   OPENROUTER_API_KEY=your_key_here
#   ALLOWED_ORIGINS=["http://localhost:3000"]

uvicorn app.main:app --reload --port 8000
# API docs: http://localhost:8000/api/docs
```

Frontend

```
cd frontend
npm install

# frontend/.env.local:
#   NEXT_PUBLIC_API_URL=http://localhost:8000

npm run dev                    # http://localhost:3000
# (npm run build && npm start for production)
```

One-command (Docker)

```
docker-compose up --build      # brings up backend + frontend together
```


08 Sample Input & Output Demonstration

A. Natural-language maintenance query

Input (engineer, in the AI Command chat):

```
"Why is Blast Furnace-1 showing high vibration and what should I do?"
```

Routing: supervisor detects "vibration" → MAINTENANCE agent.

Output (structured, explainable):

Sensor analysis indicates Blast Furnace-1 is operating with elevated vibration (7.4 m/s²) and temperature (1,435 C), consistent with advancing bearing wear.

Recommended actions (prioritised):

1. [IMMEDIATE] Reduce load on BF-1 by 15% and inspect bearing within 6h.
2. [24 h] Replace primary thrust bearing (~8h outage, cost INR 4.2L).
3. [48 h] Full vibration audit of Sinter Plant conveyor assembly.
4. [1 week] Calibrate all pressure sensors on BOF Converter.

Confidence: 87% | Expected impact: prevents ~72h downtime, saves INR 18.5L.

B. Real-time threshold-breach alert (emitted by the live engine)

```
{
  "id": "EVT-00042-BLA",
  "type": "THRESHOLD_BREACH",
  "plant": "Plant-A",
  "unit": "Blast-Furnace-1",
  "severity": "CRITICAL",
  "message": "Blast-Furnace-1 failure probability crossed 60% -> 78.0%",
  "metric": "failure_probability",
  "value": 0.78,
  "timestamp": "2026-06-15T07:34:31Z",
  "is_live": true
}
```

C. Injected demo alert (the on-stage "Simulate" button)

```
POST /api/demo/inject { plant: 'Plant-D', type: 'CRITICAL_FAILURE' }
-> spikes the unit's sensors (vibration 8.9, temp 1580, FP 0.91),
   pushes a CRITICAL event to the live feed, and the dashboard +
   digital twin recolor that asset to red within one poll cycle.
```